

제4회 KAIST ACM-ICPC 모의대회 해법

문제 난이도

쉬움 : A B D J

보통 : F H L

어려움 : C I K

매우 어려움 : E G

A. 문제 순서는 난이도 순이 아닙니다

```
int sum = 0, nanido;  
for (int i = 0; i < n; i++) {  
    scanf("%d", &nanido);  
    if (i+1 == nanido) sum++;  
}  
// 이번 대회 난이도?  
// 12  
// 1 2 8 4 11 7 12 6 9 3 10 5
```

B. 좋은 문제 집합

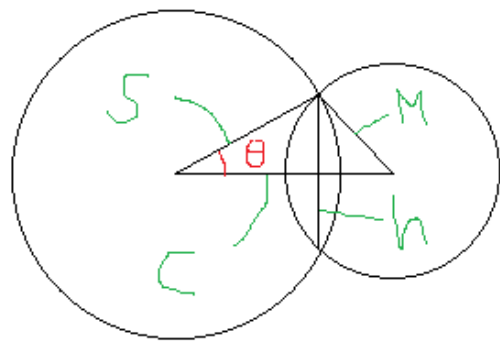
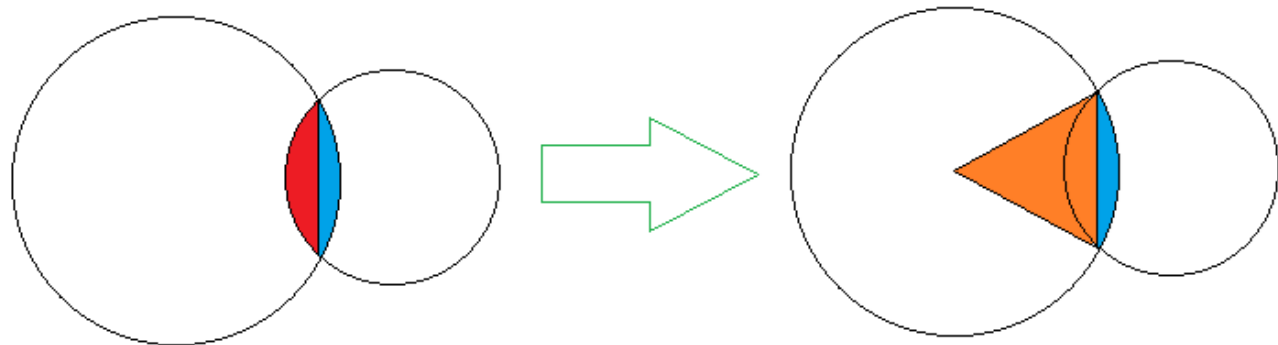
1. 모든 팀은 적어도 한 문제를 풀고 만점은 없다

- 주어지는 X_i 가 0 혹은 N이 아니면 된다

2. $V[i] = 0, 1$

i 번 문제가 누군가에게 풀렸으면 1, 아니면 0
모든 참가자를 고려했을 때 $V[i]=0$ 인게 존재하는지 확인

J. 달과 그림자



코사인
제2법칙

D. 용감한 쿠키군의 폭탄 해체

1. BFS로 풀기

1. 한 번도 방문하지 않은 정점에서 시작하여 BFS를 수행하고, 이 BFS를 수행하는 중에 만난 점들은 시작점과 같은 그룹임.
2. 이렇게 그룹들을 구하고, 가장 큰 그룹을 찾으면 끝!

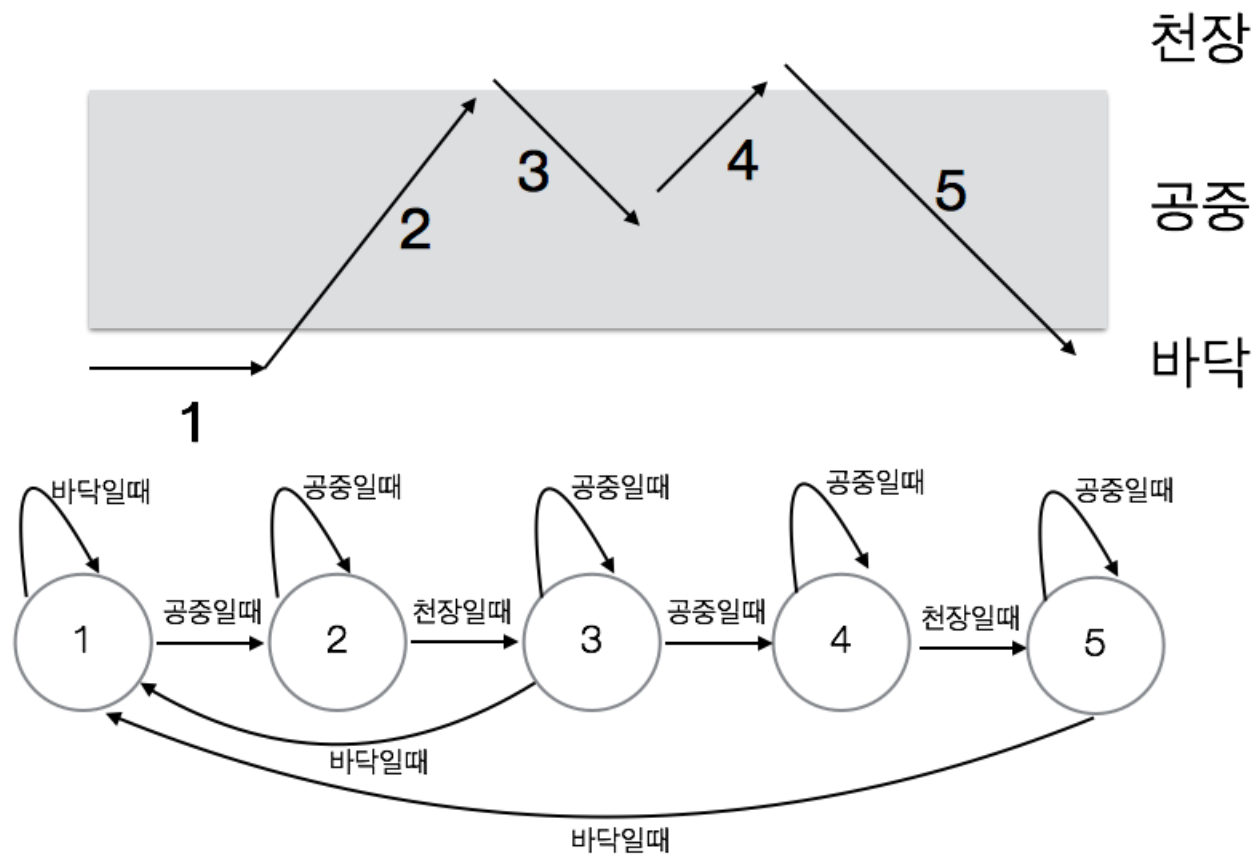
2. Union-Find로 풀기

1. 연결되어 있으면 같은 집합!
2. 전부 Union하고, 1.2와 같이 가장 큰 그룹을 찾으면 끝.

L. 시내 도로 교통 조사

1. $N \leq 100$. N 이 작다. Floyd algorithm!
 1. $\text{edge}(a, b)$ = 교차로 a 교차로 b 사이, 도로 길이
 2. $\text{dist}[a][b]$ = a 에서 b 로 가는 최단 거리
 3. $\text{dist}[a][a] = 0$
2. A 에서 B 로 가는 최단 경로?
 1. A 에서 B 로 바로 가기
 2. A 에서 ?를 거쳐서, ?에서 B 로 가기
3. $\text{isUniqueWay}(A, B) = \text{true}$ 이기 위한 조건
 - 1) A 에 연결된 모든 교차로 C 에 대해, $\text{edge}(A, C) + \text{dist}[C][B] == \text{dist}[A][B]$ 인 C 가 하나만 있다.
 - 2) $\text{isUniqueWay}(C, B)$ 가 true이다.

코기런



출제자 : 이준성

코기런

- $D[i][j][1,2,3,4,5]$: (i, j) 에 코기가 도달할 수 있으면 1, 아니면 0
- Dynamic Programming!

F. 스타포스 강화

최종적으로 가능한 장비 상수 : 0~65535

답을 정해두고, M번 강화하여 답을 만들 수 있는지를 본다.

1. 뽑기 버튼을 누른다.
2. 1에서 나온 카드와 지금까지 뽑았으며 사용하지 않은 카드를 bitwise-or한다.
3. 2와 우리가 정한 답 x 를 bitwise-and 했을 때 x 가 나온다?
 1. 바로 강화!
 1. 어차피 우리가 만들 답은 x 고, 더 뽑아봤자 낭비다!
 2. 아니면 1로 간다.
4. 총 M회 강화하면 끝! 1이 가장 많은 답을 찾자!
 - 참고로 "버리기" 버튼은 의미가 없음.

F. 스타포스 강화

참고!

==는 &보다 우선순위가 더 높음.

$(x \& \text{now}) == x$ (맞음)

$x \& \text{now} == x$ (틀림)

C. 문자열 비교

1. 문자열 파싱 → (문자열, 반복횟수, 시작인덱스)

$e(abcd)^{10}f \rightarrow ("e", 1, 0), ("abcd", 10, 1), ("f", 1, 41)$

$eab(cdab)^9 \rightarrow ("e", 1, 0), ("a", 1, 1), ("b", 1, 2), ("cdab", 9, 3)$

2. 겹치는 부분만 비교

(문자열1 = "abcd", 10, 1) vs (문자열2 = "cdab", 9, 3)

ab**cd**

cdab

처음으로 겹치는 인덱스 찾음 = 3

len(문자열1) * len(문자열2) 만큼만 비교

3. 전체적인 시간복잡도 $O(\text{len}(A) * \text{len}(B))$

I. 주차장

5x5 의 주차장. 각 차가 주차장 안에 있을 때 가능한 위치의 수? ≤ 4

최대 9개의 차!

4^9 ? 262,144!! 작다!

I. 주차장

약 20만개의 상태!

1. Memoization + DFS
2. Dynamic Programming
3. BFS

각 상태에서는 최대 $2*N$ 개의 상태로 뻗어 갈 수 있다

P.S. 근데 생각해보면 5x5에선 가능한 경우가 별로 없어요. 구현하자. (실제 러시아워 게임은 6x6)

K.xorequation

정보가 하나도 없을 때 : $ans = 2^n$

i, j, x 의 정보가 들어왔을 때 정보를 분할해보면

1 $i-1$ 0 and 1 j (0 XOR x)

1 $i-1$ 1 and 1 j (1 XOR x) 2가지로 표현 가능하다.

1 ~ $i-1$ 의 XOR가 0이면 1 ~ j 의 XOR가 (0 XOR x)가 되어야 한다! 1 ~ $i-1$ 의 XOR가 1일때도 마찬가지

K.xorequation

$(i-1, 0)$ 과 $(j, 0 \text{ XOR } x)$ 를 잇는 edge

$(i-1, 1)$ 과 $(j, 1 \text{ XOR } x)$ 를 잇는 edge로 표현 가능

edge가 이어져 있으면 하나의 노드로 취급한다

Union&Find, BFS등을 이용 가능

이 때, $(i, 0)$ 과 $(i, 1)$ 이 이어져 있으면 -1

그렇지 않을 경우 $2^{((\text{노드의 개수}-2)/2)}$ 가 답

E. 쿠키를 먹자

어린이들이 울지 않으려면 어린이들이 앉은 구간이 항상 연속적으로 유지되어야 한다.

1) 1의 위치가 주어진 경우

- 연속된 구간이 1개 이하로 유지된다.
- 정해진 의자에 앉는 어린이들만 순서대로 보면서 현재 구간에 대해서 왼쪽에 몇 명, 오른쪽에 몇 명을 앉혀서 연속된 구간을 유지하는 경우의 수를 곱해 나간다.
- 불가능 하면 해당 구간을 invalid 처리한다.

E. 쿠키를 먹자

2) 1의 위치가 주어지지 않은 경우

- 처음으로 정해진 의자에 앉는 어린이가 구간의 끝점이 되는 연속된 구간 2개가 만들어 진다.

- 이후에는 1)과 마찬가지로 각 구간에 대해서 처리해 주고, 마지막에 더해서 출력하면 된다.

G.Thief

NP다이나믹의 응용.

상태를 leaf노드의 방문상태로 두면
bfs를 통해 $2^L * N$ 에 풀수있음.

L : leaf노드의 수 ≤ 18

N : 정점의 수 ≤ 35